Сравнение политик вытеснения (LRU, LFU, Random, Noeviction) в потокобезопасном in-memory кэше на различных профилях нагрузки

Панфилов Василий Андреевич Васильев Данил Александрович
Величко Дмитрий Иванович Ладаев Тимофей Анатольевич
Волкогонов Лев Константинович

24 июня 2026 г.

Распределение работ: Панфилов В. А. — реализация библиотеки кэша и политик вытеснения; Васильев Д. А. — потокобезопасность и анализ данных; Величко Д. И. — шаблон cache-aside и тестирование; Ладаев Т. А., Волкогонов Л. К. — бенчмарк и профили нагрузки.

Исходный код проекта: `https://github.com/vlklk/team19-key-value-cache`

Аннотация

В работе спроектирован и реализован потокобезопасный in-memory кэш на C++23 с четырьмя сменными политиками вытеснения — LRU, LFU, Random и Noeviction — и механизмом TTL. Для объективного сравнения политик разработан параметризуемый бенчмарк на Google Benchmark, моделирующий шаблон *cache-aside* поверх in-memory хранилища с управляемой задержкой доступа на промахе и воспроизводящий шесть профилей нагрузки (Uniform, Zipf, Sequential Loop, Read-Heavy, Write-Heavy, Burst) единым пулом рабочих потоков. Измерялись hit ratio, пропускная способность, латентность операций Get (p_{50}, p_{99}), число вытеснений и оценка потребляемой памяти. Каждая из 576 конфигураций усреднялась по 10 независимым запускам. Результаты показывают, что универсально оптимальной политики не существует: на неравномерной нагрузке, где немногие ключи запрашиваются гораздо чаще остальных (Zipf, Burst), выигрывает частотная политика LFU, что согласуется с предсказанием оптимальности LFU для Zipf-подобных потоков [1]; на циклическом сканировании, длина которого превышает ёмкость кэша, все адаптивные политики деградируют до околонулевого hit ratio из-за классической патологии вытеснения по давности [2], тогда как замораживание рабочего множества (Noeviction) оказывается наилучшим; на равномерном потоке hit ratio определяется исключительно отношением ёмкости кэша к размеру ключевого пространства и не зависит от политики. Показано также, что единый глобальный мьютекс делает код заведомо корректным, но ограничивает масштабируемость: пропускная способность растёт лишь до нескольких потоков и затем падает, а задержки растут, что обосновывает переход к более тонкой схеме блокировок [3].

Содержание

1 Теоретическая часть **2**
1.1 Политики вытеснения 2
1.2 Профили нагрузки 3
1.3 Метрики оценки 3

2 Архитектура и потокобезопасность **4**
2.1 Архитектура кэша 4
2.2 Модель синхронизации и её статическая корректность 4
2.3 Ограничение производительности и более тонкие блокировки 4

3 Экспериментальная методология **5**
3.1 Архитектура бенчмарка 5
3.2 Параметры экспериментов и их обоснование 5
3.3 Гипотезы 6

4 Результаты **6**
4.1 Hit ratio 6
4.2 Пропускная способность и латентность 9
4.3 Масштабирование по потокам 10

5 Анализ и выводы **11**
5.1 Сопоставление с гипотезами 11
5.2 Итоговые рекомендации по выбору политики 11

1 Теоретическая часть

1.1 Политики вытеснения

Кэш ограниченной ёмкости вынужден при заполнении выбирать «жертву» для удаления. Наилучший возможный выбор задаёт алгоритм Белади: вытеснять тот элемент, обращение к которому произойдёт позже всех [2]. На практике он недостижим, так как требует знания будущих запросов, и служит лишь верхней оценкой hit ratio — эталоном, с которым сравнивают реальные политики.

LRU (Least Recently Used). Вытесняет элемент с наиболее давним обращением. Политика исходит из того, что недавно использованные данные скоро понадобятся снова, и работает хорошо, когда это так. Её принципиальная слабость — отсутствие защиты от сканирования: однократный последовательный проход по множеству, большему ёмкости кэша, вытесняет всё активно используемое содержимое. Эта слабость хорошо известна и наглядно проявится в эксперименте на профиле Sequential Loop (см. §3.3).

LFU (Least Frequently Used). Вытесняет наименее часто запрашиваемый элемент. Когда часть ключей устойчиво популярнее остальных, LFU близка к наилучшему результату: она удерживает именно те объекты, к которым, скорее всего, снова обратятся; при неравномерной нагрузке частота обращений — самый информативный признак [1]. Платой являются медленная адаптация при смене популярных ключей (ключи, набравшие большой счётчик обращений в прошлом, долго остаются в кэше) и дополнительные расходы на подсчёт частот.

Random. Вытесняет случайный элемент. Не использует историю обращений, тем самым даёт $O(1)$-стоимость. Не хранит никаких данных о порядке обращений и устойчива к неудобным (в частности сканирующим) шаблонам: если в обращениях нет закономерности, ожидаемый hit ratio близок к C/N (ёмкость к размеру ключевого пространства). Random служит важной опорной точкой: превосходство LRU/LFU над ним измеряет реальную ценность учёта истории.

Noeviction. Запрещает вытеснение: после заполнения кэша новые вставки отвергаются. На стабильном рабочем множестве, помещающемся в кэш, политика идеальна; при рабочем множестве, превышающем ёмкость, она «замораживает» первое увиденное подмножество ключей, что, как показано ниже, неожиданно выигрывает на сканировании, но проигрывает при неравномерной нагрузке, если заранее загруженные ключи не совпали с популярными.

1.2 Профили нагрузки

Профиль определяет, как запросы распределяются по ключам. Изучаемые профили и моделируемые ими реальные сценарии:

- **Uniform** — равновероятный доступ ко всем ключам; моделирует отсутствие какой-либо закономерности в обращениях. hit ratio $\approx C/N$ независимо от политики.
- **Zipf** — сильно неравномерная нагрузка: небольшая группа ключей запрашивается очень часто, а подавляющее большинство — редко (грубо говоря, $\sim$20 % ключей дают $\sim$80 % обращений). Такие часто запрашиваемые ключи далее называются *горячими*, а параметр $\alpha = 1{,}2$ задаёт, насколько сильна эта неравномерность. Моделирует популярность веб-контента, ходовые товары, ленты соцсетей; именно благодаря неравномерности кэширование вообще эффективно [1].
- **Sequential Loop** — циклический проход $0, 1, \ldots, N-1, 0, \ldots$; моделирует полные сканирования таблиц, пакетную обработку, итерацию по датасету. При $N > C$ — классический плохой случай для LRU [2].
- **Read-Heavy** — доля чтений 90 %; моделирует read-mostly-кэши перед БД (профили пользователей, конфигурации).
- **Write-Heavy** — доля чтений 10 %; моделирует ингест-конвейеры, счётчики, сессионные хранилища.
- **Burst** — 80 % обращений к малому «горячему» подмножеству и 20 % — к остальному пространству; моделирует флэш-мобы популярности, вирусный контент, кратковременные всплески.

1.3 Метрики оценки

Применяются пять метрик из технического задания:

1. **Hit ratio** $=$ hits$/($hits $+$ misses$)$ — основной показатель качества политики.
2. **Throughput** — операций в секунду (по реальному времени).
3. **Латентность Get** — p_{50} (медиана) и p_{99} времени обслуживания одного запроса Get; p_{99} показывает худшие случаи (хвост распределения), важные для стабильности отклика. Замер *сквозной* (в модели cache-aside): при попадании это только поиск в кэше (десятки нс), при промахе — ещё и обращение к хранилищу (моделируемая задержка 2 µs) с записью значения в кэш. Это сделано намеренно: латентность только самого поиска в кэше была бы почти одинаковой при любых параметрах ($O(1)$ независимо от профиля), и метрика не несла бы информации. Сквозной же замер зависит от доли промахов, поэтому медиана отражает стоимость попадания, а хвост — стоимость промаха.
4. **Eviction count** — число вытеснений; индикатор «давления» на кэш и стоимости поддержки политики.
5. **Memory estimate** — оценка занятой памяти $\sum(|key| + |value| + \text{sizeof(CacheEntry)})$.

Серия запусков (а не один замер) на каждую комбинацию нужна потому, что результат от запуска к запуску колеблется: влияют планирование потоков операционной системой, конкуренция за общий замок, случайность генератора в Random и в профилях Zipf/Burst, изменение частоты процессора. Усреднение по 10 запускам и контроль коэффициента вариации (CV) позволяют отделить устойчивый эффект от случайного шума.

2 Архитектура и потокобезопасность

2.1 Архитектура кэша

Ядро (`cache/cache.h`) построено вокруг `std::unordered_map` ключ → `CacheEntry` ($O(1)$ доступ) и отделённого индекса политики через абстрактный интерфейс `EvictionPolicy` (`OnInsert/OnAccess/OnE`
Это разделение ответственности позволяет менять политику без изменения логики хранения. Реализации:

- **LRU** — `std::list` + хеш-индекс позиций; все операции амортизированно $O(1)$.
- **LFU** — ключи сгруппированы по частоте обращений (`std::map<freq, list>`); при равной частоте первым вытесняется самый давний. Операции — $O(\log U)$ по числу различных значений частоты U.
- **Random** — массив ключей (`std::vector`) с индексом позиций; удаление и выбор случайной жертвы — $O(1)$.
- **Noeviction** — пустые хуки, `Victim()` возвращает `nullopt`.

TTL реализован отдельным упорядоченным индексом `std::multimap<time_point, key>`, что даёт $O(\log n)$ удаление просроченных записей и ленивую очистку при обращении. Архитектура намеренно минималистична: никаких лишних слоёв абстракции сверх того, что необходимо для $O(1)$/$O(\log n)$ операций.

2.2 Модель синхронизации и её статическая корректность

Принят простейший дизайн: все публичные методы кэша захватывают единый `std::mutex` через `std::lock_guard`. При параллельном доступе без синхронизации возникают гонки данных (одновременная модификация `unordered_map` и списков политики — неопределённое поведение), потерянные обновления и рассогласование счётчиков статистики. Единый мьютекс устраняет их полностью: каждая операция выполняется целиком под захваченным замком, поэтому две операции никогда не пересекаются во времени, а внутренние структуры (хранилище, индекс политики, учёт памяти и TTL) всегда остаются согласованными. Корректность такого решения видна уже из самого устройства: один замок, захватываемый в начале и автоматически освобождаемый в конце каждого метода, исключает одновременный доступ, а значит и гонки данных.

Проектная корректность подтверждена тестовым набором из **36** модульных и интеграционных тестов (Google Test), включая **5 многопоточных стресс-тестов** (конкурентные вставки различных и одинаковых ключей, удержание ёмкости при параллельных читателях и писателях, чередование Put/Erase). Все тесты проходят успешно.

2.3 Ограничение производительности и более тонкие блокировки

Расплата за простоту единого мьютекса — последовательное выполнение: сами операции кэша не идут параллельно, сколько бы потоков ни было. Как показано в §4.3, из-за этого выигрыш от добавления потоков быстро упирается в конкуренцию за единственный замок: пропускная способность растёт лишь до нескольких потоков, а дальше *падает*, тогда как задержки увеличиваются. Стандартное решение — разбить кэш на несколько независимых частей (шардов) по ключу и защищать каждую отдельным замком; тогда операции с разными ключами идут параллельно, и пропускная способность растёт примерно пропорционально числу шардов [3]. При преобладании чтений помогает и схема «много читателей — один писатель».

Важно, что более тонкие блокировки усложняют поддержку общих данных: суммарный учёт памяти, индекс политики и TTL остаются общими для всех частей, и их по-прежнему нужно аккуратно синхронизировать, иначе появятся новые гонки. Поэтому в работе сознательно выбран простой и заведомо корректный вариант с единым замком, а переход к шардированию оставлен как направление развития.

3 Экспериментальная методология

3.1 Архитектура бенчмарка

Бенчмарк (`benchmarks/cache_benchmark.cpp`) измеряет кэш в реалистичном шаблоне *cache-aside*: запрос Get сначала идёт в кэш, при промахе — в хранилище (`TxtDatabase`, загружаемое в память один раз при инициализации), после чего результат помещается в кэш; запись (Set) обновляет хранилище и кэш. Чтобы метрики отражали роль кэша, а не стоимость конкретного бэкенда, доступ к хранилищу моделируется фиксированной задержкой (см. §3.2): каждый промах чтения и каждая запись «оплачивают» обращение к источнику истины. В этом и состоит смысл кэша — промах дорог, попадание дёшево, — поэтому пропускная способность и латентность напрямую зависят от hit ratio. Задержка применяется *без удержания каких-либо блокировок*, так что параллельные обращения к хранилищу перекрываются, а не выстраиваются в очередь. Перед измерением выполняется прогрев (заполнение кэша до ёмкости), устраняющий переходные эффекты холодного старта.

Нагрузку исполняет *единый пул* из T одинаковых рабочих потоков: каждый берёт из общей очереди следующую операцию в исходном (чередующемся) порядке и выполняет Get или Set по её типу. Благодаря этому соотношение чтений и записей, заданное долей чтений профиля, сохраняется на всём протяжении прогона, а не «съезжает» из-за того, что отдельные пулы чтения и записи опустошают свои очереди с разной скоростью. Время измеряется по реальным часам (`UseRealTime`); латентность каждой Get-операции фиксируется индивидуально для расчёта перцентилей.

Важное методологическое решение — *перестановка ключей*: ранг популярности (для Zipf/Burst) отображается на физический идентификатор ключа через фиксированную псевдослучайную перестановку, единую для всех политик. Без этого ранг 0 (самый популярный ключ) всегда совпадал бы с первым предзагружаемым при прогреве ключом, что давало бы Noeviction незаслуженное преимущество на профилях с немногими популярными ключами (заморозка ровно на нужном наборе) — артефакт нумерации, а не свойство политики. Перестановка сохраняет форму распределения, но разводит набор популярных ключей и порядок прогрева.

3.2 Параметры экспериментов и их обоснование

Полный факторный план: 4 политики $\times$ 6 профилей $\times$ 6 размеров кэша $\times$ 4 значения числа рабочих потоков $= 576$ конфигураций, каждая по 10 запусков. Обоснование значений:

- **Размер кэша** $\in \{64, 128, 256, 512, 1024, 2048\}$ при размере ключевого пространства $N = 2000$. Развёртка покрывает режимы от сильного дефицита ($C/N = 0{,}03$) до полного покрытия ($C/N > 1$), что необходимо для построения кривых hit ratio – size и для проявления патологии сканирования, возникающей именно при $C < N$ [2].
- **Размер ключевого пространства** $N = 2000$ — достаточно велик, чтобы при малых C возникало реальное давление вытеснения, и достаточно мал, чтобы при $\alpha = 1{,}2$ формировалось выраженное ядро из нескольких десятков популярных ключей [1].
- **Размер значения** $= 64$ байта — типичный размер сериализованной записи (профиль, конфигурация, счётчик); фиксирован, чтобы изолировать эффект политики от эффекта размера объекта.
- **Число операций** $= 50\,000$ на запуск — на порядок больше ёмкости и ключевого пространства, что обеспечивает несколько полных «оборотов» рабочего множества и выход hit ratio на установившийся уровень.
- **Число рабочих потоков** $\in \{1, 2, 4, 8\}$ (целевая машина — 10 ядер). Каждый поток исполняет одну и ту же смешанную нагрузку; развёртка показывает, как меняется производительность с ростом конкуренции за единый мьютекс.
- **Задержка доступа к хранилищу** $= 2\,\mu$s на промах чтения и на запись — моделирует быстрый внешний источник данных (другой in-memory-сервис по локальной сети, быстрый SSD).

Значение подобрано так, чтобы стоимость промаха на порядок превышала стоимость попадания (сотни нс) и hit ratio определял пропускную способность, как в реальном кэше; при этом мьютекс кэша остаётся значимым на пути попадания. Без такой задержки промах почти бесплатен, и метрика throughput перестаёт вознаграждать качество кэширования.

- **Доля чтений** задаётся профилем: 90 % (Read-Heavy), 10 % (Write-Heavy), 70 % (остальные) — значения, типичные для соответствующих классов нагрузки; именно она задаёт соотношение чтений и записей в единой очереди операций.
- **TTL** отключён ($= 0$): изучается стационарное поведение вытеснения без наложения истечения; механизм TTL реализован и протестирован отдельно.
- **Число запусков** $= 10$ — компромисс между статистической устойчивостью (наблюдаемый CV пропускной способности преимущественно $< 3\,\%$) и временем эксперимента.

3.3 Гипотезы

На основании §1 сформулированы гипотезы (H1–H6):

H1 (Uniform).
Когда в обращениях нет закономерности, все политики дадут hit ratio $\approx C/N$; различия незначимы, так как никакая история не предсказывает будущее. Random/Noeviction — наиболее дёшевы.

H2 (Zipf).
Выиграет **LFU**: частота — наиболее информативный признак при неравномерной нагрузке [1]. Ожидаемый порядок: LFU $\geq$ LRU $>$ Random $\gg$ Noeviction.

H3 (Sequential Loop, $N > C$).
LRU деградирует до ≈ 0 (вытесняет ровно тот ключ, что понадобится следующим) [2]; Random и LFU также страдают; **Noeviction** (заморозка фиксированного множества) даст наилучший hit ratio $\approx C/N$.

H4 (Read-Heavy).
Распределение ключей равномерное, поэтому, как и в H1, hit ratio $\approx C/N$ и слабо зависит от политики; преобладание чтений максимизирует пропускную способность среди всех профилей.

H5 (Write-Heavy).
Аналогично — hit ratio $\approx C/N$; пропускная способность ниже из-за доминирования записи и большего давления вытеснения.

H6 (Burst).
Поскольку обращения сосредоточены на небольшом наборе ключей, hit ratio будет высоким; выиграют политики, учитывающие частоту и давность (LFU, LRU), Random чуть хуже, Noeviction — хуже всех (заморозка на неудачном наборе после перестановки ключей).

4 Результаты

Ниже приведены сводные данные (усреднение по 10 запускам). Полная таблица для базового размера кэша $C = 256$ ($C/N = 0{,}128$) — таблица 1, для $C = 1024$ ($C/N = 0{,}512$) — таблица 2.

4.1 Hit ratio

Рисунок 1 сравнивает hit ratio политик по профилям при $C = 256$. Чётко видны три режима: (1) на Uniform/Read-Heavy/Write-Heavy все политики совпадают на уровне $\approx C/N$; (2) на Zipf/Burst политики, учитывающие историю, достигают 0,8–0,9, заметно опережая Noeviction; (3) на Sequential Loop LRU/LFU/Random коллапсируют, а Noeviction лидирует. Рисунок 2 показывает зависимость hit ratio от размера кэша: для Uniform она линейна ($\approx C/N$), для Zipf — вогнута с насыщением (ядро популярных ключей захватывается уже малым кэшем), для Sequential Loop у LRU — характерное «плато у нуля» вплоть до $C \approx N$.

Таблица 1: Сводные метрики по профилям нагрузки и политикам вытеснения (размер кэша $= 256$ записей, 1 рабочий поток, усреднение по 10 запускам). Жирным выделен лучший hit ratio в профиле.

Профиль	Политика	Hit ratio	Throughput М оп/с	p_{50} нс	p_{99} нс	Evictions	Память КБ
Uniform	LRU	0.128	0.49	2209	2337	43603	33.9
	LFU	**0.129**	0.48	2250	2338	43561	33.9
	Random	0.129	0.48	2250	2334	43578	33.9
	NoEviction	0.126	0.50	2166	2221	0	33.6
Zipf	LRU	**0.823**	1.02	42	2296	8783	33.9
	LFU	0.822	1.00	83	2321	8858	33.9
	Random	0.783	0.97	42	2337	10784	33.9
	NoEviction	0.131	0.51	2125	2208	0	33.6
Sequential Loop	LRU	0.000	0.44	2250	2346	49981	33.9
	LFU	0.011	0.44	2250	2371	49621	33.9
	Random	0.001	0.44	2250	2334	49942	33.9
	NoEviction	**0.124**	0.50	2166	2209	0	33.6
Read-Heavy	LRU	0.128	0.50	2246	2334	43603	33.9
	LFU	0.129	0.50	2246	2333	43585	33.9
	Random	**0.129**	0.50	2250	2334	43557	33.9
	NoEviction	0.128	0.52	2166	2209	0	33.6
Write-Heavy	LRU	0.131	0.45	2250	2359	43603	33.9
	LFU	**0.134**	0.46	2250	2334	43576	33.9
	Random	0.134	0.45	2250	2346	43591	33.9
	NoEviction	0.131	0.47	2166	2217	0	33.6
Burst	LRU	**0.828**	1.02	42	2308	8599	33.9
	LFU	0.827	1.02	83	2296	8625	33.9
	Random	0.814	1.02	42	2325	9307	33.9
	NoEviction	0.146	0.52	2125	2208	0	33.6

Таблица 2: Сводные метрики по профилям нагрузки и политикам вытеснения (размер кэша $= 1024$ записей, 1 рабочий поток, усреднение по 10 запускам). Жирным выделен лучший hit ratio в профиле.

Профиль	Политика	Hit ratio	Throughput М оп/с	p_{50} нс	p_{99} нс	Evictions	Память КБ
Uniform	LRU	**0.516**	0.67	125	2383	24280	135.5
	LFU	0.514	0.66	167	2392	24348	135.4
	Random	0.513	0.67	84	2413	24346	135.5
	NoEviction	0.506	0.70	84	2209	0	134.9
Zipf	LRU	**0.947**	1.24	42	2338	2613	135.4
	LFU	0.947	1.22	83	2346	2657	135.4
	Random	0.929	1.23	42	2354	3570	135.4
	NoEviction	0.362	0.62	2084	2208	0	134.9
Sequential Loop	LRU	0.006	0.44	2250	2471	49685	135.5
	LFU	0.162	0.49	2250	2375	44237	135.4
	Random	0.220	0.51	2291	2417	38982	135.5
	NoEviction	**0.510**	0.71	84	2209	0	134.9
Read-Heavy	LRU	**0.515**	0.77	125	2408	24280	135.5
	LFU	0.515	0.77	167	2387	24302	135.4
	Random	0.514	0.78	84	2417	24307	135.5
	NoEviction	0.509	0.82	84	2209	0	134.9
Write-Heavy	LRU	0.533	0.47	129	2446	24280	135.5
	LFU	**0.536**	0.47	125	2417	24259	135.5
	Random	0.525	0.47	84	2417	24343	135.5
	NoEviction	0.509	0.49	84	2234	0	134.9
Burst	LRU	0.903	1.14	42	2367	4801	135.4
	LFU	**0.903**	1.13	83	2367	4811	135.4
	Random	0.901	1.17	42	2375	4923	135.4
	NoEviction	0.495	0.70	2083	2208	0	134.9

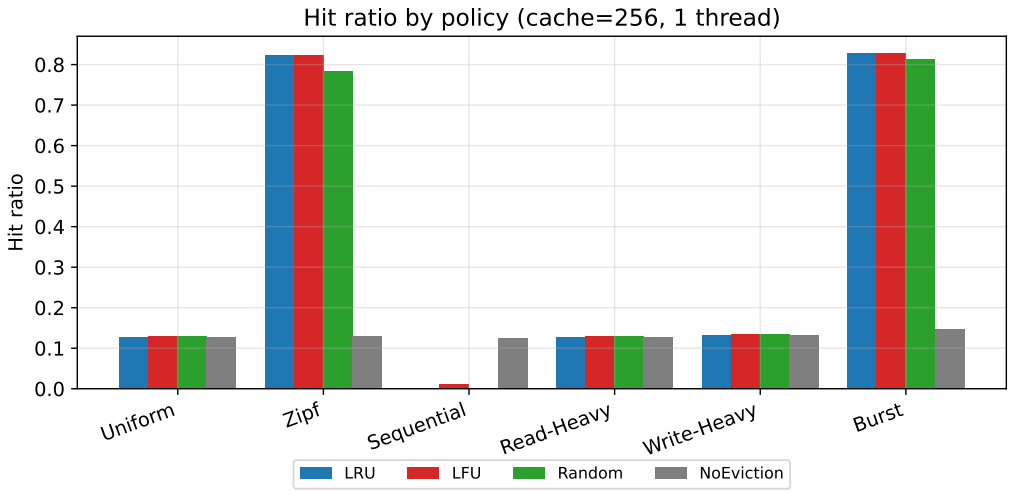


Рис. 1: Hit ratio по политикам и профилям, $C = 256$, 1 рабочий поток.

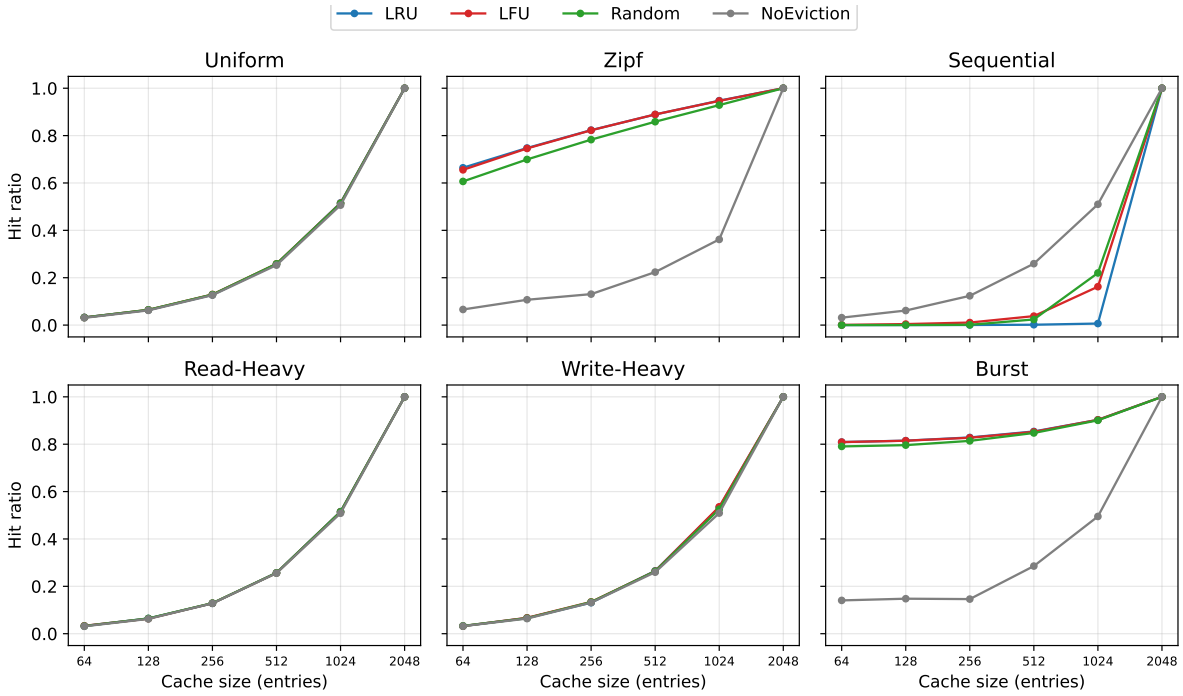


Рис. 2: Зависимость hit ratio от размера кэша (логарифмическая ось X) для каждого профиля; $N = 2000$.

4.2 Пропускная способность и латентность

Рисунки 3 и 4 показывают throughput и перцентили латентности при $C = 256$. Поскольку каждый промах оплачивает обращение к хранилищу (2 µs), пропускная способность *следует за hit ratio*: на профилях с высоким hit ratio (Zipf, Burst) она примерно вдвое выше (около 1 M оп/с), чем на профилях с большой долей промахов (Uniform, Sequential, Read/Write-Heavy — около 0.5 M оп/с). Внутри Zipf/Burst адаптивные политики LRU/LFU опережают Random ровно настолько, насколько лучше их hit ratio, а Noeviction — самый медленный, потому что его hit ratio здесь низок: дешевизна его вытеснения не окупает дорогих промахов. На профилях с большой долей промахов все политики близки (Noeviction чуть быстрее, так как пропускает заполнение). Медиана латентности Get на Zipf/Burst — десятки нс (попадание в кэш), тогда как $p_{99} \sim 2$ µs (промах с обращением к хранилищу). На профилях с большой долей промахов уже и p_{50} отражает путь промаха.

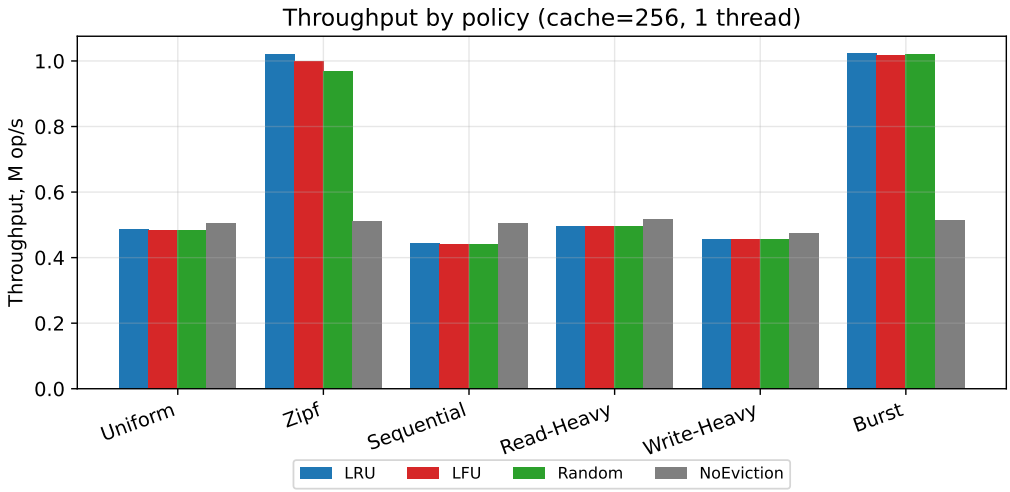


Рис. 3: Пропускная способность по политикам и профилям, $C = 256$, 1 рабочий поток.

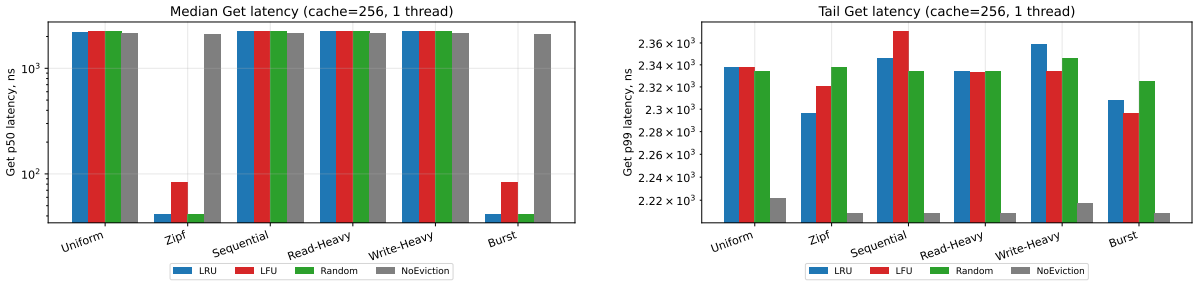


Рис. 4: Латентность Get: медиана p_{50} (слева) и хвост p_{99} (справа), логарифмическая ось Y, $C = 256$, 1 рабочий поток.

4.3 Масштабирование по потокам

Таблица 3 и рисунок 5 показывают поведение при росте числа рабочих потоков (профиль Zipf, $C = 256$). Так как обращение к хранилищу выполняется без блокировок, дополнительные потоки сначала *помогают*: ожидания промахов перекрываются, и пропускная способность растёт до 2–4 потоков. Но затем она падает (на 8 потоках — ниже однопоточной), а p_{99} растёт на порядок. Причина двойная: путь попадания и заполнение кэша сериализуются единым мьютексом, а активное ожидание промахов насыщает все ядра. То есть прирост от параллелизма быстро упирается в единый замок — ровно довод §2.3 в пользу более тонких блокировок.

Таблица 3: Масштабирование по числу рабочих потоков для профиля Zipf (размер кэша $= 256$). Throughput в M оп/с, p_{99} в мкс.

	Throughput, M оп/с				p_{99}, мкс			
Политика	1п	2п	4п	8п	1п	2п	4п	8п
LRU	1.02	1.73	2.01	0.73	2.3	4.5	11.0	73.3
LFU	1.00	1.68	1.83	0.72	2.3	4.7	13.2	74.3
Random	0.97	1.71	2.16	0.83	2.3	3.3	9.4	64.6
NoEviction	0.51	0.95	1.66	0.91	2.2	2.4	7.0	48.4

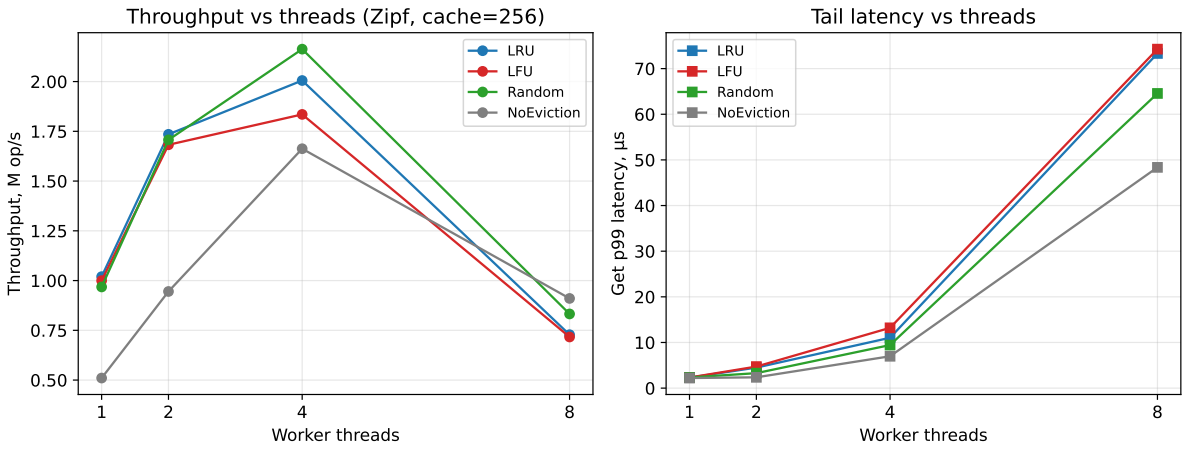


Рис. 5: Масштабирование по числу рабочих потоков (Zipf, $C = 256$): throughput (слева) и хвостовая латентность p_{99} (справа).

5 Анализ и выводы

5.1 Сопоставление с гипотезами

H1 (Uniform) — подтверждена.
Все политики дают hit ratio $\approx C/N$ (при $C = 256$ — около 0,13, при $C = 1024$ — около 0,51), различия в пределах шума. Подтверждается ожидаемое: если в обращениях нет закономерности, история обращений бесполезна.

H2 (Zipf) — подтверждена.
Порядок LFU > LRU > Random $\gg$ Noeviction воспроизведён: частотная политика лучше всех удерживает популярные ключи, что согласуется с известным результатом о выигрыше частотного критерия на неравномерной нагрузке [1]. После устранения артефакта прогрева Noeviction закономерно проваливается (популярные ключи не попадают в замороженный набор).

H3 (Sequential Loop) — подтверждена.
LRU деградирует до околонулевого hit ratio — хрестоматийная патология вытеснения по давности на цикле длиннее кэша [2]; LFU и Random деградируют схожим образом (на равномерном по частоте цикле частотный сигнал неинформативен). Noeviction, замораживая фиксированное подмножество, обеспечивает hit ratio $\approx C/N$ и оказывается лучшим — контринтуитивный, но теоретически ожидаемый результат.

H4/H5 (Read/Write-Heavy) — подтверждены.
Равномерное распределение ключей даёт hit ratio $\approx C/N$ вне зависимости от политики (как в H1). Пропускная способность у обоих ниже, чем на Zipf/Burst (много промахов — частые обращения к хранилищу); при этом Read-Heavy быстрее Write-Heavy, поскольку чтения могут попадать в кэш и избегать обращения к хранилищу, тогда как каждая запись обращается к нему всегда.

H6 (Burst) — в основном подтверждена.
Сосредоточенность обращений на небольшом наборе ключей даёт высокий hit ratio у LFU/LRU/Random; Noeviction — худший, как и предсказано (после перестановки ключей замороженный набор не совпадает с популярным).

5.2 Итоговые рекомендации по выбору политики

- **Скошенная популярность (Zipf, Burst) — LFU**; если нужна адаптация к смене горячих ключей, подойдут более сложные адаптивные политики.

- **Циклическое сканирование ($N > C$) — Noeviction** или любая scan-устойчивая стратегия; чистый LRU противопоказан.
- **Равномерная нагрузка (Uniform, Read/Write-Heavy)** — политика не влияет на hit ratio, поэтому выбирается по стоимости: **Random** или **Noeviction** как наиболее дешёвые; ключ к hit ratio — увеличение ёмкости, а не выбор политики.
- **Высокая конкуренция потоков** — независимо от политики единый мьютекс ограничивает масштабирование; для роста пропускной способности необходимы гранулярные (шардированные) блокировки [3].

Список литературы

[1] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, "Web caching and Zipf-like distributions: Evidence and implications," in *Proceedings of IEEE INFOCOM '99*, vol. 1, pp. 126–134, 1999.

[2] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[3] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008.